

Урок 11. Корутины. Dispatcher

25 февраля 2020

В этом уроке рассмотрим основные виды диспетчеров. Также поговорим о том, что происходит под капотом корутин при использовании диспетчера.

В прошлых уроках мы уже немного использовали диспетчеры и знаем, что корутина при запуске ищет в своем контексте диспетчер, чтобы получить поток для выполнения работы.

Давайте разбираться какие диспетчеры нам доступны.

Default

Если корутина не находит в своем контексте диспетчер, то она использует диспетчер по умолчанию. Этот диспетчер представляет собой пул потоков. Количество потоков равно количеству ядер процессора.

Он не подходит для IO операций, но сгодится для интенсивных вычислений.

В качестве примера запустим 6 корутин на 4-ядерном процессоре.

```
1 val scope = CoroutineScope(Dispatchers.Default)
2
3 repeat(6, {
4     scope.launch {
5         log("coroutine $it, start")
6         TimeUnit.MILLISECONDS.sleep(100)
7         log("coroutine $it, end")
8     }
9 })
```

Одновременно запускаем все корутины. Они пойдут к диспетчеру, но он сможет предоставить им только 4 потока. Так что кому-то явно придется подождать.

Смотрим логи:

```
19:56:52.658 coroutine 0, start [DefaultDispatcher-worker-1]
19:56:52.658 coroutine 2, start [DefaultDispatcher-worker-3]
19:56:52.658 coroutine 3, start [DefaultDispatcher-worker-4]
19:56:52.658 coroutine 1, start [DefaultDispatcher-worker-2]
19:56:52.761 coroutine 0, end [DefaultDispatcher-worker-1]
19:56:52.761 coroutine 3, end [DefaultDispatcher-worker-4]
19:56:52.761 coroutine 2, end [DefaultDispatcher-worker-3]
19:56:52.761 coroutine 1, end [DefaultDispatcher-worker-2]
19:56:52.763 coroutine 4, start [DefaultDispatcher-worker-4]
19:56:52.763 coroutine 5, start [DefaultDispatcher-worker-3]
19:56:52.865 coroutine 5, end [DefaultDispatcher-worker-3]
19:56:52.865 coroutine 4, end [DefaultDispatcher-worker-4]
```

Корутины 0,2,3 и 1 начали работу. Диспетчер выдал им потоки DefaultDispatcher-worker 1, 3, 4 и 2. На этом свободные потоки закончились, и корутинам 5 и 4 пришлось ждать, пока потоки освободятся.

Учитывайте, что ядра могут быть не только физическими, но и [виртуальными](#).

IO

Использует тот же пул потоков, что и диспетчер по умолчанию. Но его лимит на потоки равен 64 (или числу ядер процессора, если их больше 64).

Этот диспетчер подходит для выполнения IO операций (запросы в сеть, чтение с диска и т.п.).

Выполним тот же пример, что и для Default, чтобы убедиться, что теперь всем корутинам хватит потоков.

```
1 val scope = CoroutineScope(Dispatchers.IO)
2
3 repeat(6, {
4     scope.launch {
5         log("coroutine $it, start")
6         TimeUnit.MILLISECONDS.sleep(100)
7         log("coroutine $it, end")
8     }
9 })
```

Лог:

```
19:59:49.503 coroutine 3, start [DefaultDispatcher-worker-4]
19:59:49.503 coroutine 2, start [DefaultDispatcher-worker-3]
19:59:49.503 coroutine 1, start [DefaultDispatcher-worker-1]
19:59:49.506 coroutine 4, start [DefaultDispatcher-worker-6]
19:59:49.503 coroutine 0, start [DefaultDispatcher-worker-2]
19:59:49.508 coroutine 5, start [DefaultDispatcher-worker-5]
```

```
19:59:49.608 coroutine 3, end [DefaultDispatcher-worker-4]
19:59:49.608 coroutine 0, end [DefaultDispatcher-worker-2]
19:59:49.609 coroutine 2, end [DefaultDispatcher-worker-3]
19:59:49.608 coroutine 4, end [DefaultDispatcher-worker-6]
19:59:49.610 coroutine 5, end [DefaultDispatcher-worker-5]
19:59:49.615 coroutine 1, end [DefaultDispatcher-worker-1]
```

Все корутины работали одновременно и никому не пришлось ждать.

В чем разница между IO и Default?

Давайте сформулируем этот вопрос немного по-другому.

Почему Default не подходит для IO операций?

IO операции не требуют больших затрат CPU. Мы можем запустить их хоть 10 или 100 одновременно, чтобы они суммарно выполнились быстрее. И при этом мы не особо нагрузим процессор.

Если мы будем запускать такие операции в Default диспетчере, то мы тем самым ограничим количество одновременно выполняемых операций. Напомню, что у этого диспетчера количество потоков равно количеству ядер процессора. В итоге все IO операции выстроятся в очередь и будут ждать. С точки зрения производительности в этом нет смысла.

Почему IO не подходит для тяжелых вычислительных операций?

Вычислительные операции требуют больших затрат CPU. Если мы параллельно запустим много таких операций, то CPU будет перегружен и не сможет выполнять другие задачи. Поэтому IO диспетчер тут не подходит, ему доступно слишком много потоков.

Если мы будем запускать такие тяжелые операции в Default диспетчере, то они выстроятся в очередь и будут ждать, а CPU не будет перегружен.

Executor

Мы можем создать свой диспетчер на базе Executor.

Запустим все тот же пример, но в качестве диспетчера используем Executor с одним потоком.

```
1  val scope = CoroutineScope(
2      Executors.newSingleThreadExecutor().asCoroutineDispatcher()
3  )
4
5  repeat(6, {
6      scope.launch {
7          log("coroutine $it, start")
8          TimeUnit.MILLISECONDS.sleep(100)
9          log("coroutine $it, end")
10     }
11 })
```

6 корутин придут к диспетчеру, у которого есть только один поток.

Логи:

```
20:26:39.502 coroutine 0, start [pool-1-thread-1]
20:26:39.604 coroutine 0, end [pool-1-thread-1]
20:26:39.605 coroutine 1, start [pool-1-thread-1]
20:26:39.706 coroutine 1, end [pool-1-thread-1]
20:26:39.707 coroutine 2, start [pool-1-thread-1]
20:26:39.808 coroutine 2, end [pool-1-thread-1]
20:26:39.808 coroutine 3, start [pool-1-thread-1]
20:26:39.911 coroutine 3, end [pool-1-thread-1]
20:26:39.911 coroutine 4, start [pool-1-thread-1]
20:26:40.013 coroutine 4, end [pool-1-thread-1]
20:26:40.013 coroutine 5, start [pool-1-thread-1]
20:26:40.114 coroutine 5, end [pool-1-thread-1]
```

Корутинам пришлось выстраиваться в очередь и выполняться последовательно.

Main

Main диспетчер запустит корутину в основном потоке.

Когда начинаешь изучать корутины, то кажется, что в них можно выполнять только тяжелые фоновые задачи и никакого Main потока тут быть не может. Но это не так. Корутина запросто может выполняться в Main потоке. Просто надо делать это правильно и со знанием дела, чтобы не заблокировать этот поток.

Если вы используете RxJava, то вспомните, что там тоже есть Scheduler, который работу в Main поток переводит. С корутинами все примерно также.

Чтобы понять, зачем может понадобиться корутина в Main потоке, сделаем пример с вызовом suspend функции. Напомню, что suspend функция приостанавливает код корутины, выполняет свою работу в фоновом потоке, а затем возобновляет корутину. Поток корутины при этом не блокируется. О том, как это происходит, я очень подробно рассказывал в первых уроках этого курса.

Напишем свою простую suspend функцию, которая эмулирует получение данных с сервера.

```
1 private suspend fun getData(): String =
2     suspendCoroutine {
3         thread {
4             TimeUnit.MILLISECONDS.sleep(3000)
5             it.resume("Data")
6         }
7     }
```

Если вам нужен живой пример, то вы можете использовать библиотеку Retrofit. Она умеет http-запросы оборачивать в suspend функции.

Данные, которые будут получены от этой функции, я хотел бы сразу отобразить на экране. Пусть для этого у меня есть некий метод updateUI. Т.е. код будет выглядеть так:

```
1 val data = getData()
2 updateUI(data)
```

Здесь мы вызываем suspend функцию, чтобы получить данные, а затем эти данные используем для обновления UI. И это накладывает на нас два ограничения.

Во-первых, этот код должен быть запущен в корутине, т.к. getData - это suspend функция, и вне корутины ее не запустить.

Во-вторых, этот код должен быть запущен в Main потоке, т.е. метод updateUI обновляет данные на экране, а это можно сделать только в Main потоке.

Объединяем эти ограничения и получается, что нам нужна корутина, выполняющая свой код в Main потоке:

```
1 val scope = CoroutineScope(Dispatchers.Main)
2
3 scope.launch {
4     val data = getData()
5     updateUI(data)
6 }
```

Код внутри launch выполнится в Main потоке. Когда мы вызываем метод getData, поток не будет заблокирован. Но и код не пойдет дальше, пока данные не будут получены. Suspend функция приостанавливает выполнение кода, не блокируя поток. Когда она сделает свою работу, выполнение кода возобновится. Полученные данные мы передадим в updateUI.

Основной смысл в том, что в корутине, которая выполняется в Main потоке, мы можем спокойно писать обычный (типичный для Activity или фрагментов) код, который работает с UI. Но при этом мы можем в этом же коде вызывать suspend функции, которые будут асинхронно получать данные с сервера или БД. И нам не нужны будут колбэки и переключения потоков. Все это скрыто под капотом корутин, а наш код выглядит чище и лаконичнее.

Чтобы пример выглядел полноценно, я приведу полный код Activity:

```
1 class MainActivity : AppCompatActivity() {
2
3     private val scope = CoroutineScope(Dispatchers.Main)
4     private var job: Job? = null
5
6     override fun onCreate(savedInstanceState: Bundle?) {
7         super.onCreate(savedInstanceState)
8         setContentView(R.layout.activity_main)
9         findViewById<View>(R.id.btnRun).setOnClickListener { onRun() }
10    }
11
12    private fun onRun() {
13        if (job?.isActive == true) return
14        job = scope.launch {
15            val data = getData()
16            updateUI(data)
17        }
18    }
19
20    private fun updateUI(data: String) {
21        findViewById<TextView>(R.id.label).text = data
22    }
23
24    private suspend fun getData(): String =
25        suspendCoroutine {
26            thread {
27                TimeUnit.MILLISECONDS.sleep(3000)
28                it.resume("Data")
29            }
30        }
31
32    override fun onDestroy() {
33        super.onDestroy()
34        scope.cancel()
35    }
36
37 }
```

По нажатию на кнопку Run мы получаем данные из suspend функции и отображаем их в TextView.

На что здесь надо обратить внимание:

- 1) Не забываем вызов `scope.cancel()` в `onDestroy`. В этом конкретном примере он ни на что не повлияет. Но предполагается, что вместо `getData` с `sleep`-заглушкой, мы используем реальную `suspend` функцию, которая умеет корректно реагировать на отмену корутины.
- 2) Перед тем, как запустить корутину, выполняется проверка, что она не в работе в данный момент. Это позволяет избежать лишних запусков.

Как используется диспетчер в корутине

Мы не рассмотрели еще один диспетчер - `Unconfined`. Чтобы понять принцип его работы, необходимо знать, как именно корутина использует диспетчер для запуска. В первых уроках курса я рассказывал про `Continuation` и про его взаимодействие с `suspend` функциями. Вкратце напомним о чем шла речь.

Когда мы просим билдер создать корутину, он создает `Continuation`. Этот `Continuation` содержит код, который корутина должна выполнить. Т.е. тот самый код, который мы указываем в блоке `launch { ... }`. Этот код делится на блоки оператором `switch`. Точки разделения - `suspend` функции.

Билдер выполняет метод `continuation.resume`, и, тем самым запускает корутину. Процесс выполнения пошел. Когда дело доходит до `suspend` функции, то `Continuation` запускает эту функцию, передает туда себя и на этом заканчивает выполнение текущей ветки в `switch`.

`suspend` функция выполняет свою работу в фоновом потоке. По завершению она вызывает `continuation.resume` и передает туда результат работы, тем самым возобновляя работу `Continuation` в его следующей ветке `switch`.

Диспетчеры немного меняют эту схему. Вернее, они ее расширяют. Когда билдер создает `Continuation`, то он оборачивает его в `DispatchedContinuation` и туда же передает диспетчер корутины. Т.е. обертка `DispatchedContinuation` знает, какой код (`Continuation`) и в каком потоке (`Dispatcher`) необходимо выполнить.

Именно эту обертку использует билдер, чтобы стартовать выполнение кода корутины. Т.е. билдер вызывает не `continuation.resume` напрямую, а `dispatchedContinuation.resume`. А уже в этом методе диспетчеру поручается выделить поток и вызвать в нем `continuation.resume`. В результате код `Continuation` (он же код корутины) выполняется в потоке диспетчера корутины.

С `suspend` функцией все аналогично. Она при запуске получает не `Continuation`, а `DispatchedContinuation`. По завершению своей работы она запускает `dispatchedContinuation.resume`, что приводит к запуску `continuation.resume` в потоке корутины.

Давайте сделаем пару примеров, чтобы проследить всю эту цепочку.

В `suspend` функцию добавим логи. Первый лог покажет из какого потока была запущена эта функция. А второй - в каком потоке она выполняет свою работу.

```
1 private suspend fun getData(): String =
2     suspendCoroutine {
3         log("suspend function, start")
4         thread {
5             log("suspend function, background work")
6             TimeUnit.MILLISECONDS.sleep(1000)
7             it.resume("Data!")
8         }
9     }
```

Запустим корутину с диспетчером `Default`.

```
1 val scope = CoroutineScope(Dispatchers.Default)
2
3 scope.launch() {
4     log("start coroutine")
5     val data = getData()
6     log("end coroutine")
7 }
```

В корутине вызываем `suspend` функцию и пишем логи. Первый лог покажет в каком потоке выполняется код корутины до запуска `suspend` функции, а второй - после.

Логи:

```
start coroutine [DefaultDispatcher-worker-1]
suspend function, start [DefaultDispatcher-worker-1]
suspend function, background work [Thread-5]
end coroutine [DefaultDispatcher-worker-3]
```

Билдер создает `dispatchedContinuation` и вызывает его метод `resume`. Диспетчер находит свободный поток (`DefaultDispatcher-worker-1`) и отправляет туда `Continuation` на выполнение. Код `Continuation` выполняется и вызывает `suspend` функцию. При вызове `suspend` функции никакой диспетчер не используется. Идет обычный вызов метода. По логам (`suspend function, start`) мы видим, что `suspend` функция была вызвана в потоке корутины. На этом завершилось выполнение первой части `Continuation`.

Далее `suspend` функция создает отдельный поток (`Thread-5`) и уходит туда, чтобы выполнить свою фоновую работу и не блокировать поток корутины (`DefaultDispatcher-worker-1`). По завершению этой работы, она вызывает `dispatchedContinuation.resume`, чтобы возобновить выполнение корутины. Диспетчер находит свободный поток (`DefaultDispatcher-worker-3`) и отправляет туда `Continuation` на продолжение выполнения. `Continuation` в этом потоке выполняет код из следующей ветки `switch`.

Резюмируя все вышесказанное, получается такая схема.

Первый вызов `Continuation` был выполнен билдером. Диспетчер отправил его в поток `DefaultDispatcher-worker-1`. В этом потоке выполнялась первая часть кода и произошел запуск `suspend` функции.

```
1 log("start coroutine")
2 val data = getData()
```

`Suspend` функция по окончании работы выполнила второй вызов `Continuation`. Диспетчер отправил его в поток `DefaultDispatcher-worker-3`. В этом потоке выполнялась оставшаяся часть кода.

```
1 | log("end coroutine")
```

В итоге получилось так, что поток начала работы корутины не совпадает с потоком окончания. Это выглядит крайне необычно и запутанно, если не знать, что именно происходит под капотом. Но вы теперь знаете, что `suspend` функция может сменить поток вашей корутины. И не забывайте, что функции `join()` и `await()`, которые мы проходили в уроке про билдеры, это тоже `suspend` функции.

Но есть и хорошая новость. Смена потока не произойдет, если мы используем диспетчер `Main`. Вернее, если мы используем диспетчер с одним потоком. У него просто нет выбора. Все выполнения кода он будет отправлять в свой единственный поток.

Unconfined

Теперь, когда мы знаем, как диспетчер используется корутинами, мы можем обсудить диспетчер `unconfined`.

Для этого вернемся к `DispatchedContinuation`. Напомню, что именно его метод `resume` запускает и продолжает выполнение кода корутины. Внутри этого метода диспетчер корутины отправляет `Continuation` корутины на выполнение в нужный поток.

Но перед тем, как отправить код на выполнение, выполняется проверка - а надо ли тут использовать диспетчер. Для этого вызывается метод `dispatcher.isDispatchNeeded`. Т.е. диспетчер сам решает, должен ли он использоваться. Большинство диспетчеров отвечают на этот вопрос утвердительно. Но есть диспетчер, который добровольно отказывается. Это диспетчер `Unconfined`.

У диспетчера `Unconfined` метод `isDispatchNeeded` возвращает `false`. Это приводит к тому, что при старте и возобновлении выполнения кода `Continuation` не происходит смены потока.

Т.е. при старте корутина выполняется в том потоке, где был вызван билдер, который эту корутину создал и запустил. А при возобновлении выполнения из `suspend` функции, корутина выполняется в потоке, который использовался в `suspend` функции для выполнения фоновой работы. Ведь именно в этом потоке мы вызываем `continuation.resume`.

Чтобы убедиться в этом, сделаем тот же пример, что и ранее, но используем диспетчер `Unconfined`.

```
1 | val scope = CoroutineScope(Dispatchers.Unconfined)
2 |
3 | scope.launch() {
4 |     log("start coroutine")
5 |     val data = getData()
6 |     log("end coroutine")
7 | }
```

Логи:

```
start coroutine [main]
suspend function, start [main]
suspend function, background work [Thread-2]
end coroutine [Thread-2]
```

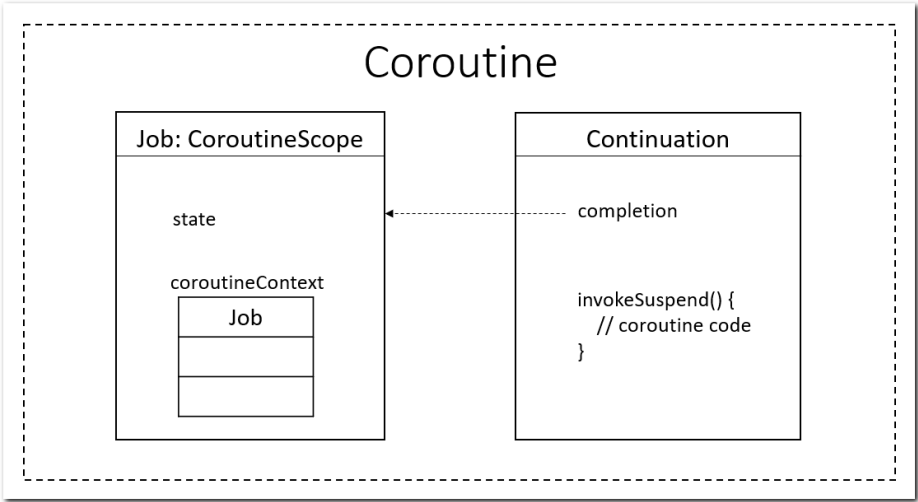
Билдер корутины был вызван в обычном коде `Activity`, т.е. в `Main` потоке. При старте корутины диспетчер `Unconfined` не стал отправлять ее в отдельный поток. Выполнение продолжилось в `Main` потоке и по логам (`start coroutine`) мы видим, что корутина начала свою работу в этом потоке.

Далее в этом же потоке произошел вызов `suspend` функции (`suspend function, start`). `Suspend` функция создала отдельный поток (`Thread-2`) для своей работы и ушла работать туда (`suspend function, background work`). Когда она закончила, она в этом же потоке вызвала метод `resume`, чтобы продолжить выполнение корутины. Диспетчер `Unconfined` не стал отправлять корутину в отдельный поток, поэтому корутина продолжила (`end coroutine`) свое выполнение в потоке `Thread-2`.

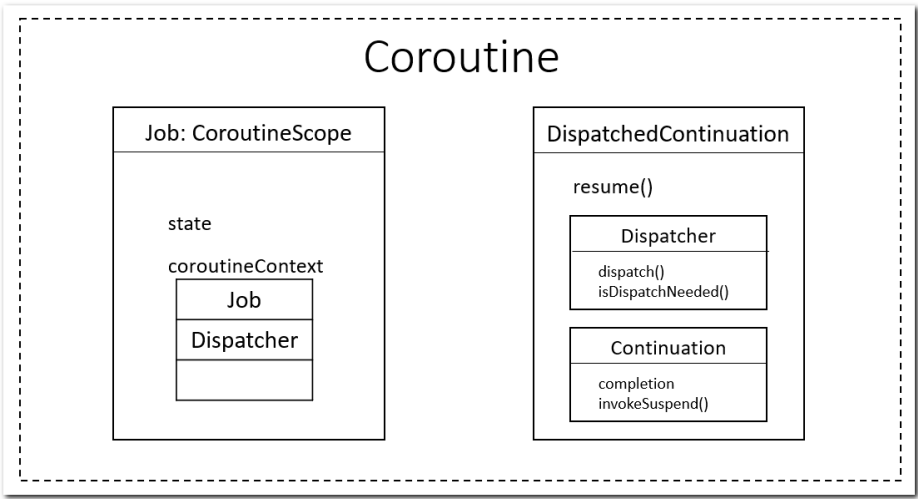
Схема

Обновим схему корутины.

Последняя версия выглядела так:



Добавим в эту схему диспетчер и DispatchedContinuation:



Continuation остается тем же, но уходит внутрь DispatchedContinuation. Здесь же находится и диспетчер, который был взят из контекста джоба.

Когда билдер стартует (или suspend функция возобновляет) работу корутины, он вызывает метод DispatchedContinuation.resume. В этом методе выполняется проверка isDispatchNeeded и, если диспетчер нужен, то вызывается его метод dispatch, который в нужном потоке стартует (продолжает) работу Continuation.